

Multi-Agent Vedic Astrology RAG Application

Project Documentation — Solution Kit

Table of Contents

- 1. The Solution We Built
- 2. Node Reference Table
- 3. Node Deep Dives
- 4. Key Design Decisions
- 5. Agent Strategy Comparison
- 6. Prompt Library
- 7. Alternatives Considered
- 8. Technologies Used
- 9. API Reference
- 10. Evaluation Framework

1. The Solution We Built

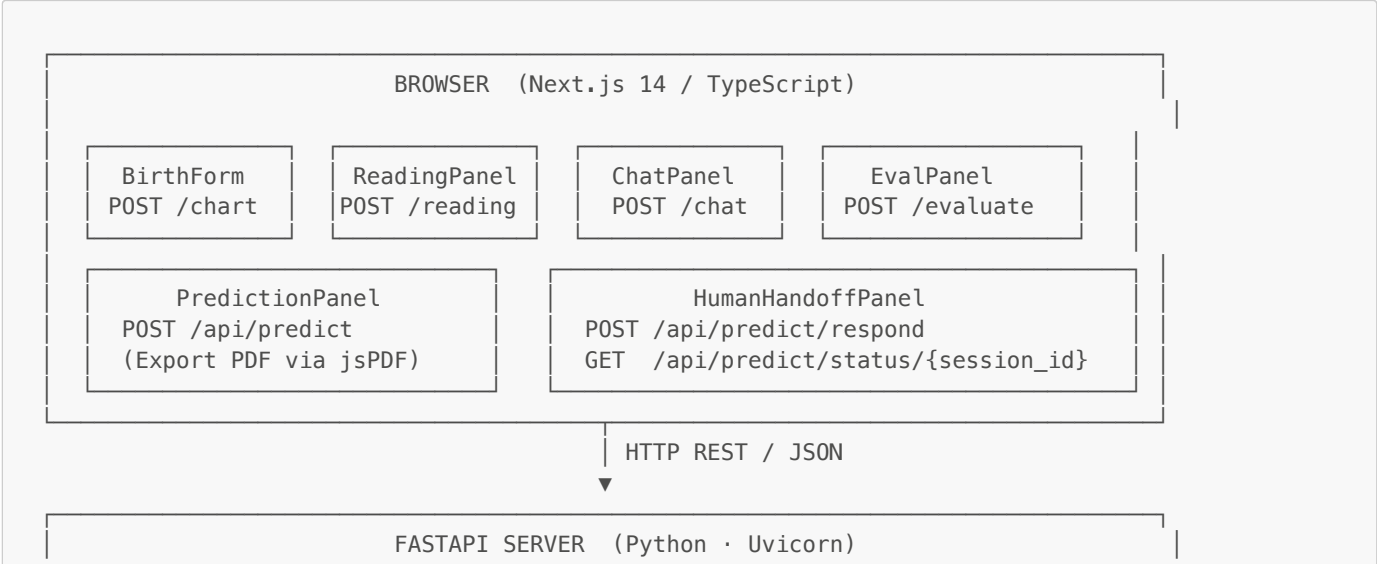
The Multi-Agent Vedic Astrology RAG Application is a full-stack platform built on two complementary AI pipelines sharing a single FastAPI backend and ChromaDB vector store.

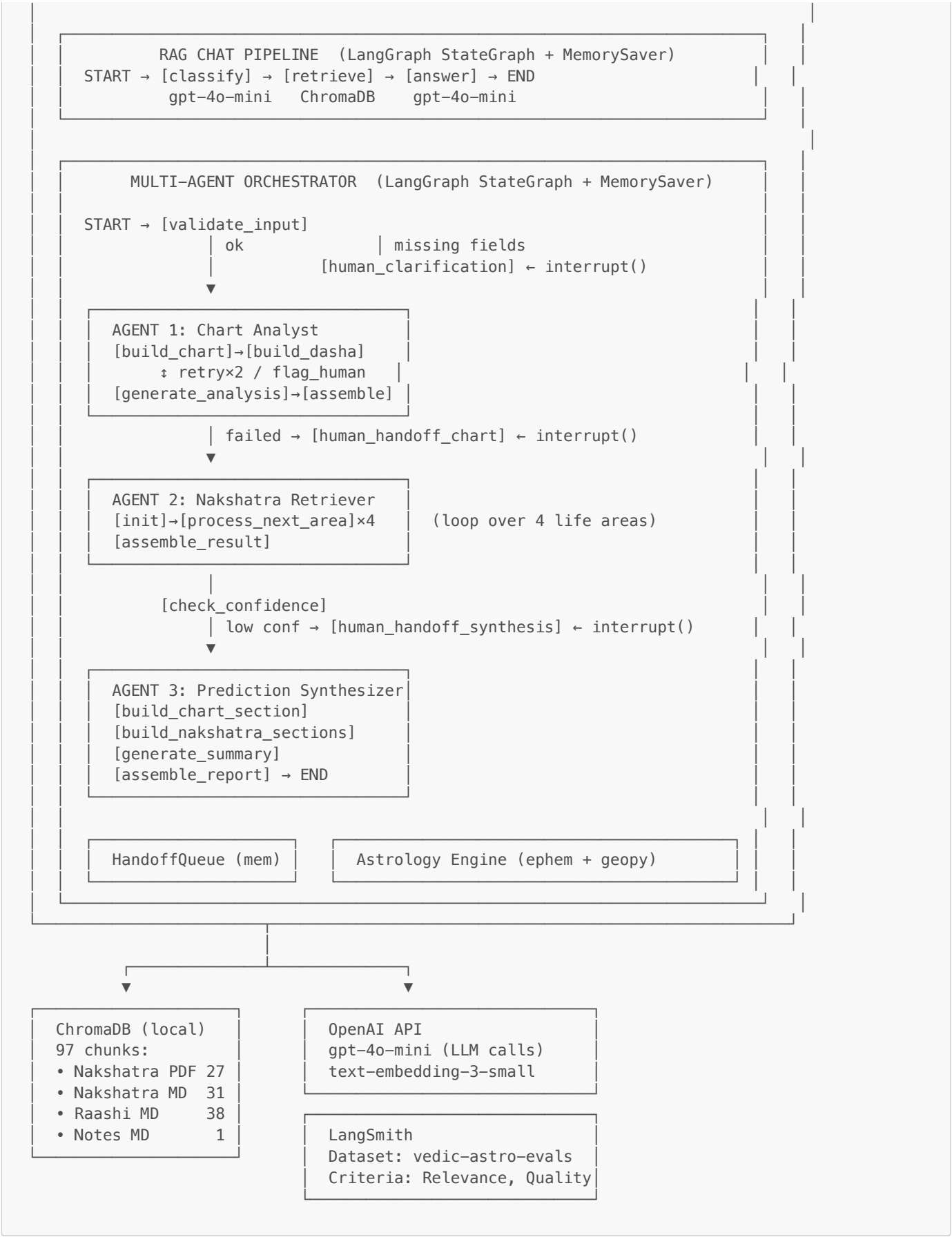
RAG Chat Pipeline — A LangGraph three-node graph (`classify` → `retrieve` → `answer`) that answers Vedic astrology questions grounded in a curated knowledge base of 97 document chunks. The classifier routes each query to the right source filter (`nakshatra` / `raashi` / `chart_specific` / `general`) before retrieval.

Multi-Agent Prediction Pipeline — A LangGraph orchestrated pipeline of four agents that accepts birth details and produces a structured personal prediction report. The Orchestrator validates input and sequences three specialist agents. The Chart Analyst builds the Vedic birth chart and Vimshottari Dasha timeline. The Nakshatra Retriever fetches nakshatra-specific passages for four life areas from ChromaDB. The Prediction Synthesizer merges everything into a final report with an executive summary, agent attribution badges, and a dasha timeline. Human handoff via LangGraph `interrupt()` pauses the graph at any of three checkpoints when the pipeline needs clarification.

Architecture Overview

The diagram below illustrates the full architecture. The user enters birth details into the Next.js frontend. The FastAPI server hosts both LangGraph graphs. The Orchestrator runs the Orchestrator → Chart Analyst → Nakshatra Retriever → Confidence Gate → Synthesizer pipeline once per prediction request. Shared state (`OrchestratorState TypedDict`) carries birth data, agent results, error log, human handoff state, and the full step history audit trail. When the pipeline completes, the final `PredictionReport` is returned to the UI.





2. Node Reference Table

Multi-Agent Orchestrator Pipeline

Node ID	Node Name	Type	LLM Calls	Description
<code>validate_input</code>	Input Validator	Pure Python	0	Checks that <code>birth_date</code> , <code>birth_time</code> , and <code>birth_place</code> are all present. Sets <code>needs_human=True</code> and surfaces a clarification question if any field is missing.
<code>request_human_clarification</code>	Human Clarification	Interrupt	0	Calls <code>interrupt(question)</code> to pause the graph when required birth fields are missing. Resumes after the user submits missing details via <code>POST /api/predict/respond</code> .
<code>run_chart_analyst</code>	Chart Analyst Delegate	LLM Agent (sub-graph)	1–2	Delegates to the Chart Analyst sub-graph. Builds the Vedic birth chart, calculates Vimshottari Dasha periods, and generates a chart overview text. Writes <code>ChartAnalysisResult</code> to <code>OrchestratorState["chart_result"]</code> .
<code>human_handoff_chart</code>	Chart Handoff	Interrupt	0	Calls <code>interrupt(question)</code> when the Chart Analyst sub-graph fails after 2 retries. Prompts the user to verify birth date format, time, and city name.
<code>run_nakshatra_retriever</code>	Nakshatra Retriever Delegate	LLM Agent (sub-graph)	4	Delegates to the Nakshatra Retriever sub-graph. Processes 4 life areas in a loop — retrieves ChromaDB passages for each and generates an interpretive section. Writes <code>NakshatraAnalysisResult</code> to <code>OrchestratorState["nakshatra_result"]</code> .
<code>check_confidence</code>	Confidence Gate	Conditional Edge	0	Reads <code>low_confidence_count</code> from <code>NakshatraAnalysisResult</code> . If ≥ 2 sections have fewer than 2 retrieved passages, triggers <code>human_handoff_synthesis</code> . Otherwise proceeds directly to synthesizer.
<code>human_handoff_synthesis</code>	Synthesis Handoff	Interrupt	0	Calls <code>interrupt(question)</code> when confidence gate fires. Asks user for supplementary context about the low-confidence life areas before synthesis.
<code>run_synthesizer</code>	Synthesizer Delegate	LLM Agent (sub-graph)	1	Delegates to the Prediction Synthesizer sub-graph. Merges chart and nakshatra results, generates an executive summary, and assembles the final <code>PredictionReport</code> with agent attribution.

Total LLM calls (happy path): ~6 per prediction run (1 chart analysis + 4 section readings + 1 executive summary). Wall-clock time scales with OpenAI API latency for each LLM call.

RAG Chat Pipeline

Node ID	Node Name	Type	LLM Calls	Description
<code>classify</code>	Query Classifier	LLM Agent	1	Classifies the user's message as <code>nakshatra</code> , <code>raashi</code> , <code>chart_specific</code> , or <code>general</code> . Uses chart context and conversation history to resolve possessive references ("my moon", "my lagna").
<code>retrieve</code>	Knowledge Retriever	Pure Python	0	Runs a ChromaDB similarity search filtered by query type (<code>source</code> metadata field). Falls back to unfiltered search if fewer than 2 results. Enriches short follow-up queries with the last AI message for context.

Node ID	Node Name	Type	LLM Calls	Description
answer	Answer Generator	LLM Agent	1	Generates a grounded answer from retrieved passages, chart context, and conversation history. Distinguishes Raashi (moon sign) from Lagna (ascendant) explicitly in the system prompt.

Total LLM calls per chat turn: 2 (1 classify + 1 generate).

3. Node Deep Dives

3.1 Chart Analyst (Agent 1)

The Chart Analyst is the entry point of the multi-agent pipeline. It receives birth data from the orchestrator, builds the full Vedic birth chart using the `ephem`-based astrology engine, computes Vimshottari Dasha periods from the Moon's nakshatra position, and generates a text overview of the chart configuration.

Key implementation details

- `build_birth_chart` tool wraps `astrology_engine.build_chart()` — geocodes the birth place with `geopy`, resolves timezone with `timezonefinder`, then computes planetary positions with `ephem`.
- `calculate_vimshottari_dasha` tool determines the ruling planet at birth from `_NAKSHATRA_LORDS[]`, computes the elapsed fraction of the first dasha from Moon's degree within its nakshatra, and projects up to 9 consecutive dasha periods forward up to 30 years.
- On `build_birth_chart` failure, the sub-graph retries once. On a second failure, it calls `flag_for_human_review` and sets `needs_human=True`. The orchestrator then triggers `human_handoff_chart`.
- The sub-graph uses conditional edges — `route_after_chart()` reads `chart_raw["success"]` and `retry_count` to decide between `build_dasha`, `retry_chart`, and `flag_human`.

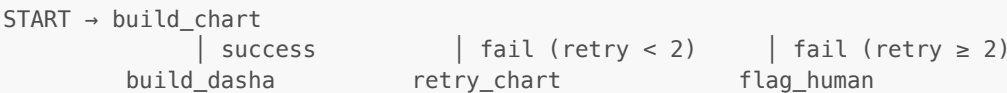
ChartAnalystState TypedDict

```
class ChartAnalystState(TypedDict):
    birth_data: dict
    chart_raw: Optional[dict]          # raw output from build_birth_chart tool
    dasha_raw: Optional[dict]          # raw output from calculate_vimshottari_dasha tool
    analysis_raw: Optional[dict]       # raw output from generate_section_reading tool
    error_log: List[str]
    retry_count: int
    needs_human: bool
    human_question: Optional[str]
    result: Optional[dict]             # final ChartAnalysisResult as dict
```

ChartAnalysisResult output schema

```
class ChartAnalysisResult(BaseModel):
    chart_data: Optional[ChartResponse]
    dasha_periods: List[DashaPeriod]
    chart_analysis_text: str
    moon_nakshatra: str                # passed to Agent 2 via OrchestratorState
    ascendant: str
    error_log: List[str]
    success: bool
```

Internal graph



```

      |
generate_analysis      | (re-enters route)      |
      |                                     assemble_result → END
      |
assemble_result → END

```

3.2 Nakshatra Retriever (Agent 2)

The Nakshatra Retriever is deliberately structured as a loop. It processes one life area per iteration — retrieving ChromaDB passages and generating an interpretation section — until all four areas are complete. This per-area isolation prevents cross-contamination between life areas and makes each section independently traceable.

Key implementation details

- `life_areas_todo` is initialised as a list of `(area_name, source_filter)` tuples: Career/nakshatra, Relationships/nakshatra, Health/nakshatra, Spirituality/general.
- Each iteration pops the first tuple, builds a targeted query ("`{nakshatra} nakshatra {area_name.lower()} Vedic astrology`"), calls `retrieve_vedic_knowledge`, and calls `generate_section_reading`.
- If fewer than 2 passages are returned, `low_confidence=True` is set for that section and `low_confidence_count` is incremented. The agent continues for all remaining areas regardless.
- The `route_continue_or_finish` conditional edge checks whether `life_areas_todo` is non-empty to decide loop-back vs `assemble_result`.

NakshatraRetrieverState TypedDict

```

class NakshatraRetrieverState(TypedDict):
    moon_nakshatra: str
    chart_summary: str
    life_areas_todo: List[tuple]      # (area_name, source_filter) remaining
    sections: List[dict]             # completed NakshatraSectionResult dicts
    failed_retrievals: List[str]
    low_confidence_count: int
    result: Optional[dict]

```

NakshatraAnalysisResult output schema

```

class NakshatraAnalysisResult(BaseModel):
    sections: List[NakshatraSectionResult] # one per life area
    failed_retrievals: List[str]
    low_confidence_count: int               # read by check_confidence gate
    success: bool

```

Internal graph (loop pattern)

```

START → init → process_next_area → (life_areas_todo non-empty?) → process_next_area
                                   | empty
                                   assemble_result → END

```

3.3 Prediction Synthesizer (Agent 3)

The Prediction Synthesizer is the final agent. It reads both `ChartAnalysisResult` and `NakshatraAnalysisResult` from the orchestrator state and assembles a single `PredictionReport`. It makes exactly one LLM call — the executive summary. All section content is assembled from the upstream agents' pre-generated text.

Key implementation details

- `node_build_chart_section` wraps the chart analysis text into a `ReportSection` with `agent_source="chart_analyst"`. If a `human_response` was captured from a prior interrupt, it is appended to the section body.
- `node_build_nakshatra_sections` converts each `NakshatraSectionResult` into a `ReportSection`. Sections with `low_confidence=True` get `confidence=0.6`; others get `1.0`. The frontend renders amber warning badges for low-confidence sections.
- `node_generate_summary` makes the single LLM call — it passes the chart summary, section headings, and low-confidence count as context.
- `node_assemble_report` builds `AgentAttribution` (records which agent produced what, plus whether human input was used) and packages the full `PredictionReport` including the `step_history` audit trail from the orchestrator.

SynthesizerState TypedDict

```
class SynthesizerState(TypedDict):
    session_id: str
    person_name: str
    chart_result: dict                # ChartAnalysisResult as dict (from Agent 1)
    nakshatra_result: dict            # NakshatraAnalysisResult as dict (from Agent 2)
    human_response: Optional[str]     # injected if human handoff occurred upstream
    chart_section: Optional[dict]
    nakshatra_sections: List[dict]
    executive_summary: Optional[str]
    result: Optional[dict]            # final PredictionReport as dict
```

PredictionReport output schema

```
class PredictionReport(BaseModel):
    session_id: str
    person_name: str
    executive_summary: str
    sections: List[ReportSection]    # chart overview + 4 life area sections
    dasha_periods: List[DashaPeriod]
    agent_attribution: AgentAttribution
    low_confidence_count: int
    human_input_used: bool
    step_history: List[str]           # full orchestrator audit trail
```

Internal graph

```
START → build_chart_section → build_nakshatra_sections → generate_summary → assemble_report →
END
```

3.4 Orchestrator

The Orchestrator owns the full pipeline lifecycle. It enforces sequential dependency gating: Agent 2 cannot start until Agent 1 writes `chart_result`; Agent 3 cannot start until both agents write their results. Human handoff uses LangGraph `interrupt()` — the graph suspends, stores the question in `HandoffQueue`, and the frontend polls `GET /api/predict/status/{session_id}` until it detects the pending question.

OrchestratorState TypedDict

```
class OrchestratorState(TypedDict):
    session_id: str
    task: str
    birth_data: dict
    person_name: str
```

```
chart_result: Optional[dict] # written by run_chart_analyst
nakshatra_result: Optional[dict] # written by run_nakshatra_retriever
final_report: Optional[dict] # written by run_synthesizer
error_log: List[str]
retry_counts: dict
needs_human: bool
human_question: Optional[str]
human_response: Optional[str]
step_history: List[str] # full audit trail
```

Orchestrator state transitions

Step	Reads from State	Writes to State
validate_input	birth_data	error_log, needs_human
request_human_clarification	human_question	human_response, needs_human=False
run_chart_analyst	birth_data	chart_result, error_log, needs_human
human_handoff_chart	human_question	human_response, needs_human=False
run_nakshatra_retriever	chart_result.moon_nakshatra	nakshatra_result
check_confidence	nakshatra_result.low_confidence_count	needs_human, human_question
human_handoff_synthesis	human_question	human_response, needs_human=False
run_synthesizer	chart_result + nakshatra_result + human_response	final_report

3.5 RAG Chat Classifier

The classifier is the first node in the RAG chat pipeline. It determines which ChromaDB source filter to apply so the retriever fetches maximally relevant passages. Misclassifying a raashi query as general would retrieve nakshatra passages, producing an incoherent answer.

Key implementation details

- The prompt includes the full list of 27 nakshatra names and 12 raashi names so the LLM can match them lexically without ambiguity.
- Possessive chart-specific rules ("my moon", "my lagna", "my ascendant") are injected into the prompt **only** when a chart context is present — this prevents the rule from firing on generic questions.
- The output is a single word. Any value outside the valid set is clamped to "general".
- The classifier reads up to the last 3 turns of conversation history so it can resolve follow-up questions like "What about health?" in context.

4. Key Design Decisions

Every significant architectural choice was made deliberately. The table below documents each decision alongside the alternative that was rejected and the reasoning.

Decision Made	Alternative Rejected	Rationale
No direct agent-to-agent calls — all data flows through OrchestratorState fields	Agents import and call each other directly	Strict boundary means each agent can be tested independently with mock state. The typed result schemas (ChartAnalysisResult, NakshatraAnalysisResult) act

Decision Made	Alternative Rejected	Rationale
		as the communication contract; any schema change is caught at the boundary.
LangGraph <code>interrupt()</code> for human handoff	Custom polling loop or webhook	<code>interrupt()</code> preserves full graph state across the pause. No state rebuild needed on resume — <code>Command(resume=answer)</code> picks up exactly where execution stopped. The three interrupt points (validation, chart failure, low confidence) are all handled identically.
All tool failures return <code>{ "success": False, "error": reason }</code> dicts	Raise exceptions from tools	Explicit failure dicts let the orchestrator state machine handle failures with conditional routing and retry logic. The chart build retries once before escalating to human handoff. Exceptions would bypass the router.
Sequential dependency gating (Agent 2 waits for Agent 1; Agent 3 waits for both)	Fan-out parallel execution	Agent 2 needs <code>moon_nakshatra</code> from Agent 1's result to build targeted retrieval queries. Agent 3 needs both agents' structured results to assemble sections. Parallelism is not possible here without a synchronisation barrier.
In-memory <code>HandoffQueue</code> dict keyed by <code>session_id</code>	Redis-backed store	Sufficient for single-instance development. The code isolates the queue in <code>human_handoff.py</code> — only that file needs to change to add Redis for multi-instance production deployment.
Existing RAG chat pipeline (<code>graph.py</code>) completely unchanged	Merge both pipelines into one graph	The two pipelines have different state shapes (<code>AstrologyState</code> vs <code>OrchestratorState</code>), different checkpointer configs, and different trigger conditions. Merging would couple unrelated logic and risk breaking the working chat pipeline.
Agent attribution per <code>ReportSection</code> (<code>agent_source</code> field)	Attribution only at report level	Per-section attribution lets the frontend render color-coded badges so the user can see which agent produced which content. Low-confidence sections are immediately identifiable without reading the entire report.
<code>MemorySaver</code> checkpointer with unique <code>thread_id</code> per session	Shared global checkpointer	Each prediction session gets its own isolated graph state. Concurrent predictions from different users do not share or overwrite each other's state.
Pydantic v2 schemas as agent communication contracts	Plain dicts passed between agents	<code>model_dump()</code> / <code>Model(**dict)</code> round-trips at every agent boundary catch schema drift immediately. <code>Field(description=...)</code> annotations also serve as self-documenting contracts.
Anti-hallucination instruction in <code>generate_section_reading</code> prompt: <code>"Do not repeat the chart summary"</code>	Trust the LLM to self-regulate	Without the explicit instruction, the LLM echoes the chart summary verbatim into every section body, producing repetitive output. The instruction forces synthesis over regurgitation.

5. Agent Strategy Comparison

Three agent architecture strategies were evaluated. The table below compares them on the dimensions that matter most for this prediction pipeline.

Dimension	Sequential Pipeline (chosen)	Parallel Specialists	ReAct Orchestrator
Speed	Slower — each agent waits for the previous result	Fastest — agents run concurrently	Variable — depends on loop count and tool call frequency

Dimension	Sequential Pipeline (chosen)	Parallel Specialists	ReAct Orchestrator
Determinism	High — fixed node order, same path every run	High — fixed branches, predictable merge	Low — agent picks its own tool sequence
LLM cost	~6 calls per run (predictable)	~6 calls but concurrent	Unbounded — agent may call tools repeatedly
Debuggability	Easy — linear step_history trace, one agent at a time	Medium — concurrent state merges can interleave	Hard — non-deterministic execution paths
Dependency handling	Native — Agent 2 reads Agent 1's output; Agent 3 reads both	Requires explicit synchronisation (aggregator node + reducer)	Agent self-manages; order not guaranteed
Human handoff	Three deterministic interrupt points	Interrupt points would need to be per-branch	Difficult — agent may skip steps or loop past the handoff trigger
Best for	Structured report with known data dependencies (this project)	Many independent parallel tasks with no shared dependencies	Open-ended, adaptive research workflows

★ Sequential Pipeline is the chosen architecture for this project because the pipeline has strict data dependencies (Agent 2 needs Agent 1's moon_nakshatra; Agent 3 needs both agents' results) that make parallelism structurally impossible without a synchronisation barrier.

6. Prompt Library

Exact prompts used in the Multi-Agent Vedic Astrology application. Both the RAG pipeline and the multi-agent pipeline use structured-context prompts — model instructions carry role definition, grounding context, and explicit constraints against hallucination.

6.1 Query Classifier (RAG Chat Pipeline)

```
prompt = f"""You are a Vedic astrology classifier. Output ONLY one word.

Known NAKSHATRA names (birth stars):
{_NAKSHATRA_NAMES} # 27 names, comma-separated

Known RAASHI names (moon signs):
{_RAASHI_NAMES} # 12 signs with Tamil/Sanskrit/English variants

Rules (apply in order):
- If the user mentions any NAKSHATRA name → output: nakshatra
- If the user mentions any RAASHI name → output: raashi
- If the user uses 'my moon', 'my sun', 'my ascendant', etc. AND chart is available → output: chart_specific
- If the user uses 'raashi' or 'moon sign' generically AND chart is available → output: raashi
- Otherwise → output: general

Conversation so far:
{history}

New query: {state["query"]}

Output ONLY one word (nakshatra / raashi / chart_specific / general):"""
```

Three things this prompt does well: (1) provides exhaustive vocabulary lists so the LLM can match by name, not inference; (2) injects chart-specific rules only when a chart exists, preventing false positives; (3) enforces single-word output, making clamping to the valid set trivial.

6.2 RAG Chat Answer Generator

```

prompt = f"""You are a warm and knowledgeable Vedic astrology guide.
Answer the user's question using ONLY the context below.
If the answer is not in the context, say so honestly – do not guess.

Important Vedic astrology distinctions:
– Raashi (moon sign) = the zodiac sign where the Moon is placed – NOT the Ascendant.
– Lagna (Ascendant) = the rising sign at the time of birth – this is separate from Raashi.
– When the user asks about their "Raashi lord", use the Moon Sign (Raashi), not the Ascendant (Lagna).

Context:
{context}

Conversation history:
{history}

User's question: {state["query"]}

Give a clear, friendly, and informative answer."""

```

The explicit Raashi/Lagna distinction is critical — without it, the LLM conflates the two concepts, which is the single most common error in Vedic astrology answers.

6.3 Section Reading Generator (Multi-Agent Pipeline)

Used by all three specialist agents via the `generate_section_reading` tool.

```

prompt = ChatPromptTemplate.from_template(
    """You are a Vedic astrology expert writing a section of a personal prediction report.

Topic / Life Area: {topic}

Chart Summary:
{chart_summary}

Relevant Vedic Passages:
{passages}

Write a clear, warm, and specific 3–5 sentence interpretation for "{topic}" based on this chart
and the passages above.
Address the person directly. Focus only on this life area. Do not repeat the chart summary.

Section body:"""
)

```

The **"Do not repeat the chart summary"** instruction prevents every section from beginning with the same ascendant and moon sign boilerplate. The **"Address the person directly"** instruction produces second-person ("Your Moon in Ashwini suggests...") output rather than third-person description.

6.4 LangSmith Evaluation Prompts

Used by the `_score()` function to evaluate each logged Q&A pair.

Relevance criterion

```

You are evaluating a Vedic astrology chatbot response.

Criterion: Is the response relevant and on-topic for the Vedic astrology question asked?

Question: {question}
Response: {answer}

```

Answer with a single character: Y (passes) or N (fails). Nothing else.

Quality criterion

You are evaluating a Vedic astrology chatbot response.

Criterion: Is the response accurate, informative, and genuinely helpful as a Vedic astrology answer?

Question: {question}
Response: {answer}

Answer with a single character: Y (passes) or N (fails). Nothing else.

The single-character Y/N constraint is the most important part of these prompts — it eliminates ambiguous partial scores and makes the pass/fail aggregation deterministic.

7. Alternatives Considered

Three alternative multi-agent architectures were evaluated before settling on the sequential orchestrated pipeline. Each is documented below with trade-offs and the reason it was not selected.

7.1 Parallel Fan-Out / Fan-In

Same three specialist agents, but the orchestrator fans out Chart Analyst, Nakshatra Retriever, and Synthesizer in parallel using LangGraph's `Send()` API. An Aggregator node merges the structured results.

Advantages	Disadvantages
Faster wall-clock time — agents run concurrently	Chart Analyst and Nakshatra Retriever have a hard dependency: Agent 2 needs Agent 1's <code>moon_nakshatra</code> . Parallel execution would require a synchronisation barrier, negating the speed benefit.
Each agent branch isolated — one failure does not block others	More complex state management — needs reducers for parallel branches and an aggregator step that doesn't exist in the sequential design.
Production-ready scaling if more specialist agents are added	Harder to debug — concurrent state merges produce interleaved logs; it is difficult to determine which agent produced which state mutation.

Why not selected: The strict data dependency (`moon_nakshatra` flows Agent 1 → Agent 2 → Agent 3) makes true parallelism structurally impossible without introducing a synchronisation barrier that cancels the performance benefit.

7.2 Supervisor with Specialist Sub-Agents

A more agentic design. An LLM-based Supervisor agent receives birth data and dynamically routes work to specialist sub-agents (Chart Builder, Knowledge Retriever, Section Writer, Dasha Calculator) based on what information is still missing from the report.

Advantages	Disadvantages
Most flexible — the Supervisor can adapt its strategy if a tool fails or returns sparse data	Highest complexity to build, test, and observe — the Supervisor's routing decisions are non-deterministic
Cleaner separation of concerns — each specialist agent has a narrow, well-defined scope	Highest token cost — Supervisor reasoning loops add LLM calls with no clear upper bound
Easy to add new specialists (e.g., a transit calculator) without changing the pipeline structure	Risk of looping or over-calling tools without explicit <code>max_iterations</code> guardrails
Best fit if sources differ per dimension (RAG for features, live ephemeris API for transits)	Human handoff is difficult to implement — the Supervisor may skip the handoff node entirely or revisit it multiple times

Why not selected: The prediction pipeline has a fixed, well-defined structure — the same nodes run in the same order for every birth input. Supervisor agency would add cost and non-determinism with no benefit, and would make the three deterministic interrupt points (validation, chart failure, low confidence) much harder to implement reliably.

7.3 ReAct Orchestrator (Single Agent)

A single ReAct agent with access to all five tools (`build_birth_chart`, `retrieve_vedic_knowledge`, `generate_section_reading`, `calculate_vimshottari_dasha`, `flag_for_human_review`) orchestrates the entire pipeline in one reasoning loop, deciding which tool to call next and when to produce the final report.

Advantages	Disadvantages
Simplest graph topology — one node, no router or gating logic	Non-deterministic execution — tool call sequence and iteration count vary between runs
Agent can adapt dynamically if chart calculation fails on the first attempt	Cannot guarantee all four life areas get equal retrieval depth — agent may skip or merge areas
Fewer moving parts to wire up initially	Hard to inject targeted improvements (e.g., add retry logic for chart building) without restructuring the entire prompt
	Human handoff via <code>interrupt()</code> is difficult — the agent chooses when to call <code>flag_for_human_review</code> , so the three structured handoff points become unpredictable

Why not selected: The structured report requires exactly five report sections, a dasha timeline, and agent attribution per section. A ReAct agent cannot reliably guarantee this structure without heavy prompt engineering that effectively re-implements the pipeline within the prompt, while losing the observability of explicit LangGraph nodes.

8. Technologies Used

Backend

Category	Technology	Version	Purpose
Web framework	FastAPI	≥ 0.115	REST API server, Pydantic schema validation, async lifespan
ASGI server	Uvicorn	≥ 0.30	Production-grade async server with hot-reload
Data validation	Pydantic v2	≥ 2.7	Request/response schemas, agent result contracts, inter-agent communication
Agent orchestration	LangGraph	≥ 0.2	<code>StateGraph</code> , conditional edges, <code>interrupt()</code> , <code>MemorySaver</code> , <code>Command(resume=)</code>
LLM framework	LangChain	≥ 0.3	<code>@tool</code> decorator, <code>ChatPromptTemplate</code> , <code>ChatOpenAI</code> , <code>OpenAIEmbeddings</code> , retriever interface
LLM provider	OpenAI	via <code>langchain-openai</code>	<code>gpt-4o-mini</code> (all generation + evaluation), <code>text-embedding-3-small</code> (embeddings)
Vector store	ChromaDB	≥ 0.5.5	Local persistent vector store for 97 knowledge base chunks
Astrology engine	ephem	≥ 4.1	Planetary position calculations
Geocoding	geopy	≥ 2.4	Birth place → latitude/longitude
Timezone resolution	timezonefinder	≥ 6.5	Birth place → UTC offset
Document ingestion	pypdf	≥ 4.0	PDF loader for Nakshatra corpus
Text splitting	langchain-text-splitters	≥ 0.3	Recursive character splitting (600–800 char chunks with overlap)

Category	Technology	Version	Purpose
Evaluation	LangSmith	≥ 0.2	Dataset management, experiment logging, LLM-as-judge evaluations
Environment config	python-dotenv	≥ 1.0	.env file loading for API keys

Frontend

Category	Technology	Purpose
Framework	Next.js 14 (App Router)	React SSR/CSR hybrid frontend
Language	TypeScript	Static typing for all components and API types
Styling	Tailwind CSS	Utility-first responsive UI
PDF export	jsPDF	Client-side PDF generation from prediction reports
HTTP client	Native fetch	All REST calls to FastAPI backend

9. API Reference

Existing Endpoints (unchanged)

Method	Path	Request	Response	Description
GET	/api/health	—	{ status, graph_ready, orchestrator_ready }	Liveness check
POST	/api/chart	BirthInput	ChartResponse	Build Vedic birth chart
POST	/api/reading	BirthInput	ReadingResponse	Generate AI reading from chart + RAG passages
POST	/api/analyze	BirthInput	AnalyzeResponse	Combined chart + reading in one call
POST	/api/chat	ChatRequest	ChatResponse	Multi-turn RAG chat with conversation memory
POST	/api/evaluate	EvalRequest	EvalResponse	Run LangSmith evaluation on logged Q&A pairs

New Multi-Agent Endpoints

Method	Path	Request	Response	Description
POST	/api/predict	BirthInput	PredictionReport HandoffQuestion	Run the full multi-agent pipeline. Returns a complete report or a HandoffQuestion if the graph hits interrupt().
POST	/api/predict/respond	HandoffResponse	PredictionReport HandoffQuestion	Resume a paused graph by submitting the human's answer via Command(resume=answer).
GET	/api/predict/status/{session_id}	—	HandoffQuestion { pending: false }	Poll for a pending human handoff. Frontend shows/hides HumanHandoffPanel based on this response.

10. Evaluation Framework

10.1 RAG Chat Evaluation (LangSmith)

The evaluation framework targets the existing RAG chat pipeline using an **LLM-as-judge** pattern.

How it works

1. Every chat turn is appended to `backend/chat_log.csv` via `log_chat()` — records `timestamp`, `session_id`, `question`, `answer`, `sources`.
2. `POST /api/evaluate` triggers `run_langsmith_eval()`:
 - Reads the CSV (filtered to a session if `session_id` is provided)
 - Creates (or recreates) a LangSmith dataset `vedic-astro-chat-evals`
 - Pushes each Q&A pair as an example
 - Runs `langsmith.evaluate()` with two LLM-as-judge evaluators
 - Returns per-row scores and aggregate percentages

Evaluation criteria

Criterion	Key	Scoring	Question
Relevance	<code>relevance</code>	1 (Y) / 0 (N)	Is the response relevant and on-topic for the Vedic astrology question asked?
Quality	<code>quality</code>	1 (Y) / 0 (N)	Is the response accurate, informative, and genuinely helpful as a Vedic astrology answer?

The judge model is `gpt-4o-mini` at `temperature=0`. The single-character Y/N scoring prompt is the most important constraint — it eliminates ambiguous partial scores and makes aggregation deterministic.

10.2 Multi-Agent Pipeline Observability

The orchestrator appends every step to `OrchestratorState.step_history`, which is included in the final `PredictionReport`. The frontend `PredictionPanel` renders this as a collapsible audit trail.

Example `step_history` for a full run (no interrupts)

```
validate_input: ok
run_chart_analyst: delegating to chart_analyst_agent
run_chart_analyst: success, nakshatra=Ashwini, ascendant=Aries
run_nakshatra_retriever: delegating to nakshatra_retriever_agent
run_nakshatra_retriever: 4 sections, low_confidence=0
check_confidence: confidence ok, proceeding to synthesizer
run_synthesizer: delegating to prediction_synthesizer_agent
chart_analyst: chart built
chart_analyst: dasha calculated
nakshatra_retriever: life areas retrieved
synthesizer: sections assembled
synthesizer: executive summary generated
run_synthesizer: prediction report complete
```

10.3 Confidence Gating

The `check_confidence` node implements a runtime quality gate: if `low_confidence_count` $\geq$ 2 (at least 2 of 4 life area sections retrieved fewer than 2 passages), the pipeline pauses and asks the user for supplementary context before synthesizing. This prevents the synthesizer from producing a report where the majority of content is generic fallback text rather than knowledge-base-grounded interpretation.

10.4 LangSmith Setup

Add these keys to `backend/.env` to enable evaluation and tracing:

```
OPENAI_API_KEY=sk-...
LANGCHAIN_API_KEY=ls_...
LANGCHAIN_TRACING_V2=true
LANGCHAIN_PROJECT=multiagent-vedicastro
```

Without `LANGCHAIN_API_KEY`, the evaluate endpoint returns `{ "status": "error", "message": "LANGCHAIN_API_KEY is not set..." }` rather than crashing.

How to Run

Backend

```
cd multiagent-vedicastro/backend
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
uvicorn app.main:app --reload --port 8000
```

On first startup, the ingest pipeline embeds all 97 knowledge base chunks into ChromaDB (~15–30 s). Subsequent starts load from disk instantly.

Frontend

```
cd multiagent-vedicastro/frontend
npm install
npm run dev
```

App runs at `http://localhost:3000`. Backend must be running at `http://localhost:8000`.

Using the Multi-Agent Prediction

1. Enter birth details and click **Generate Chart**
2. Scroll down to the **Multi-Agent Pipeline** panel
3. Click **Generate Full Prediction**
4. If the agent needs clarification, answer the modal and click **Submit & Resume Agent**
5. The full prediction report will appear with agent-attributed sections and the Dasha timeline
6. Click **Export PDF** to download the report

Project File Map

```
multiagent-vedicastro/
├── backend/
│   └── app/
│       ├── main.py           ← FastAPI app, 9 endpoints
│       ├── schemas.py        ← All Pydantic models
│       ├── tools.py           ← 5 @tool-decorated LangChain tools
│       ├── human_handoff.py   ← In-memory HandoffQueue
│       ├── graph.py           ← RAG chat LangGraph (unchanged)
│       ├── llm.py             ← generate_reading() helper
│       ├── astrology_engine.py ← ephem-based chart builder
│       ├── ingest.py          ← ChromaDB ingestion pipeline
│       ├── evaluate.py        ← LangSmith evaluation runner
│       └── agents/
│           ├── orchestrator_agent.py ← Top-level pipeline controller
│           └── chart_analyst_agent.py ← Agent 1
```

```
|
| |
| | └─ nakshatra_retriever_agent.py ← Agent 2
| | └─ prediction_synthesizer_agent.py ← Agent 3
| └─ knowledge_base/
|   |
|   | └─ Vedic_Astro_Corpus_pdf.pdf ← 27 Nakshatras (structural reference)
|   | └─ Nakshatra_Corpus.md ← 27 Nakshatras (detailed)
|   | └─ Raashi_Corpus.md ← 12 Moon signs
|   | └─ vedic_astrology_notes.md ← General chart/house basics
|   └─ requirements.txt
└─ frontend/
  └─ app/
    └─ page.tsx
    └─ components/
      └─ PredictionPanel.tsx ← Multi-agent report UI + PDF export
      └─ HumanHandoffPanel.tsx ← Interrupt modal
      └─ BirthForm.tsx
      └─ ChatPanel.tsx
      └─ ReadingPanel.tsx
      └─ HoroscopeWheel.tsx
      └─ EvalPanel.tsx
  └─ lib/
    └─ types.ts ← All TypeScript types
    └─ api.ts ← fetch wrappers for all endpoints
```